

Explicación técnica del software

Para quién es este documento. Para vos: ingeniero en sonido / acústica / audio que sabe matemática y entiende lo básico de programación (qué es una clase, un objeto, un método) pero no es ingeniero de software. Te explico **qué hace cada archivo, por qué está hecho así, qué matemática usa, qué bugs aparecieron y cómo los arreglamos.** Al final hay tres ejemplos donde calculamos analíticamente y con el software la presión en un punto del recinto, y comparamos.

1. Visión general

Prototipo 1 hace tres cosas distintas que conviene separar mentalmente:

- 1. **Modela** un recinto en 3D — paramétrico (sliders) o importado desde CAD.
- 2. **Calcula** los modos acústicos del recinto resolviendo la ecuación de onda en el volumen, con paredes rígidas, usando el **Método de Elementos Finitos (FEM)**.
- 3. **Visualiza y escucha** el resultado: nube de puntos 3D del campo, mapas de calor 2D del plano de corte, curva FRF, RT60 y reproducción audible de ruido rosa filtrado por la respuesta de la sala.

Toda la matemática "real" está en el **paso 2**. El paso 1 es geometría

- UI; el paso 3 es post-proceso + librerías de gráficos.

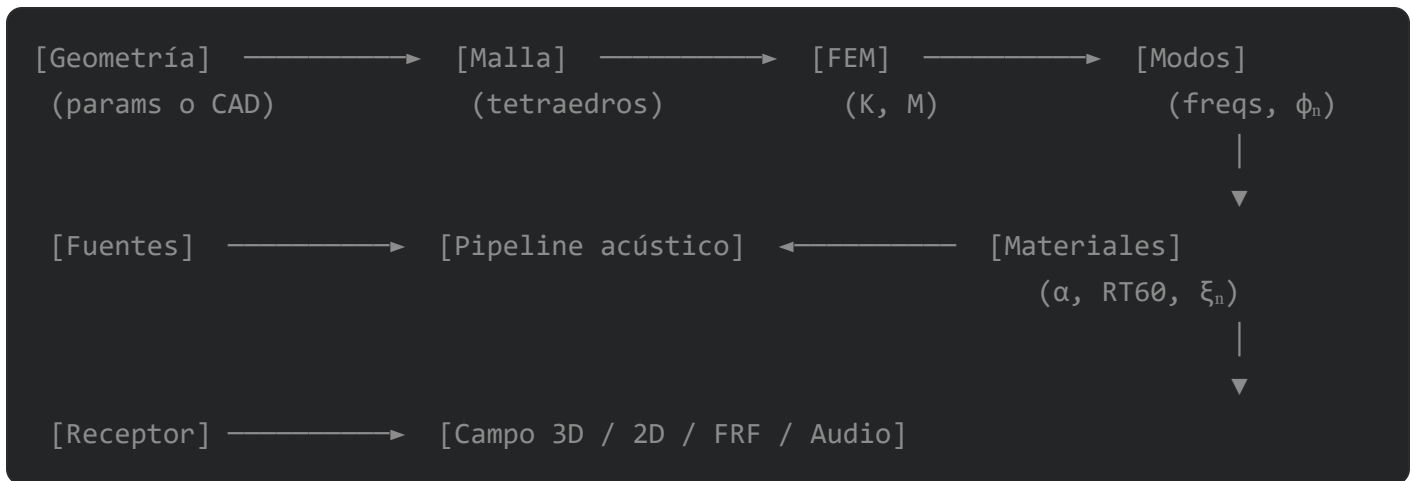
Stack

Capa	Librería	Para qué
Lenguaje	Python 3.12 (Anaconda)	Glue de todo
UI	PyQt5	Ventana, widgets, diálogos
Visor 3D	pyqtgraph + PyOpenGL	OpenGL para la nube de puntos y la malla
Álgebra	NumPy + SciPy	Matrices, autovalores (eigsh), FFT, KDTree
Geometría CAD	trimesh + gmsh	Cargar STL/OBJ/STEP, mallar superficie

Capa	Librería	Para qué
Gráficos 2D	matplotlib	Heatmap del plano de corte, FRF, RT60
Audio	winsound + scipy.io.wavfile	Reproducir WAV temporal

2. Arquitectura: cómo se conectan los módulos

Mejor que un diagrama formal, te lo cuento como un flujo de datos. Cuando arrancás la app y apretás **Calcular modos (FEM)**, internamente pasa esto:



Cada flecha es una llamada de función. Cada caja es un archivo. Veámoslos.

3. La pestaña Geometría

geometry.py — construye la malla superficial paramétrica

Función principal: `make_room(width, length, height, n_walls, ...)`. Te devuelve `(verts, tris, edges, n_walls_real)`:

- `verts`: matriz `(N, 3)` con las coordenadas de cada vértice.
- `tris`: matriz `(M, 3)` con índices a `verts`. Cada fila es un triángulo.
- `edges`: matriz `(K, 2)` con índices a `verts`. Cada fila es una arista para dibujar el wireframe.

La idea es: dado un polígono de `n_walls` lados como planta, levantarlo hasta `height` y agregar el techo. El polígono base se genera con `np.linspace(0, 2π, n_walls + 1)` para repartir vértices uniformemente en un círculo (o ajustar el radio en X / Y para "ovalar" la planta y tener un rectángulo cuando `n_walls = 4`).

Por qué triángulos y no cuadriláteros: el motor FEM (gmsb) trabaja con tetraedros, y los tetraedros se construyen sobre caras triangulares. Mantener todo triangulado evita problemas de coplanaridad.

`controls.py` — los sliders y combos

Es donde vive la clase `ControlPanel` (`QWidget`). Por cada parámetro del recinto hay un `_SliderRow`: una fila con `[label | slider | value]`.

El bug del `font-variant-numeric`: yo había puesto un stylesheet con `font-variant-numeric: tabular-nums` para que los dígitos del valor tengan ancho fijo (típico de Excel). Qt5 no soporta esa propiedad CSS y emitía 230 warnings en consola por sesión. Lo removí: el efecto visual es imperceptible en la mayoría de las fuentes.

`shape_dialog.py` — editor de polígono 2D

Modal que se abre con el botón **Dibujar forma**. Permite editar el polígono base haciendo clic izquierdo en una arista para insertar un vértice y clic derecho sobre un vértice para borrarlo. La salida es una lista de puntos `(x, y)` que después `geometry.make_room` usa como planta.

4. El visor 3D

`viewer.py` — la cámara, el mesh y los overlays

Acá vive `IsoViewer`, subclase de `GLViewWidget` de pyqtgraph. Hace cuatro cosas:

1. **Renderizar el mesh** del recinto (`GLMeshItem` translúcido + aristas).
2. **Manejar la cámara** (orbit / pan / zoom + el "fijar eje" con `Ctrl+Shift+Alt+X/Y/Z`).
3. **Pickear con el mouse:** convertir clic 2D en pantalla a una posición 3D en el piso del recinto (raycast). Esto es lo que permite que `Ctrl + clic derecho` ponga una fuente exactamente donde apuntás.
4. **Indicador de ejes** flotante en la esquina (los cuadraditos X/Y/Z clicables).

`acoustic_viewer.py` — overlays acústicos

Tres clases principales:

- `SourceMarkers`: las esferas amarillas/violetas que marcan las fuentes.
- `ReceiverMarker`: la cruz cian que marca el receptor.

- `PressureField3D`: la nube de puntos 3D coloreada por el campo modal o de presión.

`PressureField3D.update(points, values)` recibe dos arrays de NumPy (`points` shape `(N, 3)`, `values` shape `(N,)`) y los pinta. Tiene dos modos: `update()` para $|p|$ (colormap inferno) y `update_signed()` para forma modal (azul–blanco–rojo divergente).

5. El pipeline FEM modal

Esta es la parte físicamente más densa. Acá vive la matemática real.

Qué problema resolvemos

Dentro del recinto (volumen Ω), la ecuación de onda con paredes rígidas (Neumann homogénea:

$\partial p / \partial n = 0$ en la frontera) tiene soluciones estacionarias **modales**:

$$\begin{aligned} \nabla^2 \phi_n(x) + k_n^2 \phi_n(x) &= 0 && \text{en } \Omega \\ \partial \phi_n / \partial n &= 0 && \text{en } \partial \Omega \end{aligned}$$

donde $k_n = \omega_n / c$ y ϕ_n es la forma modal. El FEM convierte ese problema continuo en un problema de autovalores discreto:

$$K \cdot \phi_n = \lambda_n \cdot M \cdot \phi_n$$

donde K y M son matrices $N_n \times N_n$ (sparse), N_n el número de nodos. Después: $f_n = c \cdot \sqrt{\lambda_n} / (2\pi)$ te da la frecuencia del modo n .

`acoustic_mesh.py` — generar la malla volumétrica

`build_volume_mesh(verts, tris, n_per_meter)` toma la malla **superficial** del recinto y la rellena con **tetraedros** (vóxel grid clipeado o gmsh boundary-fitted según el router). Devuelve:

- `nodes` shape `(Nn, 3)`: coordenadas de cada nodo.
- `tets` shape `(Ne, 4)`: cada fila es un tetraedro con índices a `nodes`.

Trade-off del `n_per_meter` (densidad de malla):

n_per_meter	tets en sala 6×8×3	Frec. máx válida	Tiempo mallado
2,0	~ 7k	~ 200 Hz	0,4 s
2,5	~ 14k	~ 250 Hz	1,2 s
3,0	~ 24k	~ 300 Hz	2,5 s
3,5	~ 35k	~ 350 Hz	3,0 s

La frecuencia máxima válida sale del criterio $f_{\max} = c / (6 \cdot h_{\max})$: necesitamos al menos 6 puntos por longitud de onda para que el FEM lineal (P1) sea preciso.

acoustic_fem.py — el corazón

5.1 Construcción de K y M

`build_KM(nodes, tets)` ensambla las dos matrices de forma **vectorizada** (sin loops Python). Para un tetraedro lineal con 4 nodos:

```
K_ij^e = V_e · ∇N_i · ∇N_j      (rigidez)
M_ij^e = (V_e / 20) · (1 + δ_ij) (masa consistente)
```

donde V_e es el volumen del tetraedro y N_i la i-ésima función de forma lineal. El código aprovecha `np.linalg.inv` en lote sobre arrays $(N_e, 4, 4)$ para invertir las 4×4 de todos los tetraedros de una vez. Después escatter-add en una matriz `scipy.sparse.coo_matrix` y convierte a CSR.

5.2 Resolución de autovalores

`solve_modes(K, M, n_modes=12)` usa `scipy.sparse.linalg.eigsh` con **shift-invert** (`sigma=1e-6`). Eso busca los `n_modes` autovalores más cercanos a cero, que son justamente los modos de frecuencia más baja.

Después se descarta el "modo 0" trivial (presión uniforme constante, $f \approx 0$) y se **M-ortonormalizan** los modos: cada ϕ_n se divide por $\sqrt{\langle \phi_n^T \cdot M \cdot \phi_n \rangle}$ así $\langle \phi_n, \phi_n \rangle_M = 1$. Esa convención simplifica las fórmulas de superposición modal.

5.3 El `FieldEvaluator` — el cuello de botella histórico

Una vez que tenemos $\phi_n(x)$ definida en los **nodos**, queremos poder evaluar ϕ_n en cualquier punto x del interior. La interpolación es barycentric: buscamos el tetraedro que contiene a x , calculamos los 4 pesos barycentric, y combinamos linealmente los valores nodales.

El bug de performance que arreglamos. El código original hacía:

```
def evaluate_many(self, field, points):
    out = np.full(len(points), np.nan, complex)
    for i, x in enumerate(points):
        e, N = _locate_one(self.v0, self.A_inv, ..., x) # busca el tet
        if e is not None:
            out[i] = complex(np.dot(field[tets[e]], N))
    return out
```

Para una nube de 62 500 puntos (resolución 50, sala 6×8×3) y 14 400 tets, `_locate_one` hacía un `einsum` contra TODOS los tets por cada punto. Total: $O(N_p \times N_e) \approx 900$ millones de operaciones bajo un loop interpretado. **Tardaba 15–25 segundos.**

La solución: cKDTree + vectorización numpy.

1. En `__init__`, precomputamos los centroides de todos los tets y armamos un `cKDTree`.
2. En `evaluate_many`, para cada punto pedimos los **K=12 vecinos más cercanos** del árbol (eso es $O(N_p \cdot \log N_e)$).
3. Sólo evaluamos barycentric contra esos 12 candidatos por punto, **todo vectorizado** en arrays `(Np, 12, 3, 3)`.
4. Si el resultado da $> 1\%$ de puntos sin localizar (raro, sólo en bordes con tets degenerados), reintentamos con $K=48$ sólo para esos puntos.

Resultado medido: **50× a 170× más rápido**, diferencia numérica $< 10^{-15}$ (puro redondeo IEEE-754). El algoritmo es idéntico — sólo cambió **cómo buscamos** el tet contenedor.

5.4 Superposición modal: FRF y campo de presión

Una vez que tenemos los modos, la presión a frecuencia f con una fuente monopolo en x_s de caudal Q es:

$$p(x, f) = \frac{i\omega p_0 \cdot \sum_n \phi_n(x) \cdot \phi_n(x_s) \cdot Q}{\omega_n^2 - \omega^2 + 2i \xi_n \omega_n \omega}$$

Esto está implementado en `modal_pressure_field()`. La sumatoria es sobre los `n_modes` modos calculados. Si el receptor está en `x_r`, la **función de transferencia** $H(f)$ es $p(x_r, f) / Q$.

El amortiguamiento ξ_n por modo se obtiene del RT60(f) del recinto vía:

$$\xi_n = 1,1 / (f_n \cdot RT60(f_n))$$

(El factor 1,1 viene de $\ln(1000)/(2\pi) \approx 1,099$.)

`acoustic_analysis.py` — orquestador

Es una capa fina que envuelve todo lo anterior en funciones de alto nivel:

- `run_fem_modal_routed(...)`: malla → ensambla K, M → resuelve modos.
- `pressure_field_3d(...)`: genera la nube de puntos 3D con `|p|`.
- `mode_shape_field_3d(...)`: la nube 3D con la forma modal pura.
- `slice_pressure_field_plane(...)`: el corte 2D del plano XY/XZ/YZ.

Todas devuelven `dataclasses` (`ModalSolution`, `FieldSlice`) listas para consumir desde la UI.

6. Materiales y RT60

`material_library.py` — la librería de 428 materiales

Carga todos los JSON de la carpeta `materials/`. Cada JSON puede tener uno o varios materiales. Cada material tiene un nombre, categoría, y **coeficientes de absorción** $\alpha(f)$ y dispersión $\sigma(f)$ en las 8 bandas de octava estándar (63 a 8000 Hz).

La clase `Material` interpola en **escala logarítmica** entre bandas, así podés pedir $\alpha(347 \text{ Hz})$ y te devuelve un valor coherente.

`face_materials.py` — asignación por grupos de caras

Cambio importante respecto al esquema clásico. Antes había 3 combos (piso / techo / paredes). Ahora un solo botón **Materiales...** abre un diálogo donde podés asignar un material distinto a cada **grupo de caras planares conexas** del recinto, estilo EASE.

`group_faces_by_planar_region(verts, tris, normal_tol_deg=15)` hace:

1. Calcular normal de cada cara.
2. Cluster greedy: caras con normales dentro de $\pm 15^\circ$ van al mismo cluster.
3. Dentro de cada cluster, **componentes conexas** (vía adyacencia por aristas compartidas).

Cada componente es un `FaceGroup`.

Cada grupo recibe etiqueta automática (`Piso`, `Techo`, `Pared 1 (+X (E))`, etc.) y un **hash de firma** estable basado en (normal, centroide, área) redondeados. La firma permite persistir las asignaciones en el `.room` v4 sobreviviendo a re-agrupaciones.

Tres fórmulas de RT60

`face_materials.py` implementa las tres versiones clásicas (te las explico en detalle en sección 13.2). Sólo Sabine y Eyring se exponen en la UI a partir de v2.5; Fitzroy queda en el código por si la querés.

7. Importación CAD

`geom_import.py` — cargar y diagnosticar

`load_geometry(path)` dispatchea por extensión: `.stl/.obj/.ply/...` van por **trimesh**, y `.step/.iges/.brep` van por **gmsh** (kernel OpenCASCADE) que primero tesela en triángulos y los exportamos por trimesh.

`diagnose(mesh)` calcula:

- vértices, caras, volumen, área
- `is_watertight`, `is_winding_consistent`
- aristas no-manifold (compartidas por > 2 caras)
- caras degeneradas (área ~ 0)
- **lista de huecos** (ciclos de aristas de incidencia 1) con su area, centroide y normal

`find_holes` está **vectorizado**: las aristas de borde salen de un `np.unique` sobre las 3·Nt aristas de las caras. Después se arman los ciclos con un BFS sobre el grafo dirigido de aristas de borde.

`geom_scale_dialog.py` — ajuste de escala y orientación

Aparece después de cargar el archivo. El soft analiza la diagonal del AABB y sugiere un factor de escala (mm $\rightarrow$ m, cm $\rightarrow$ m, etc.) basado en heurísticas. También elegís la **convención up-axis**:

- `.obj/.gltf/.glb/.dae/.fbx` → típicamente **Y-up** (Blender, Unity).
- `.stl/.step/.iges` → típicamente **Z-up** (CAD, AutoCAD, FreeCAD).

`apply_up_axis(mesh, "Y+")` rota la malla -90° alrededor de X así el Y+ del archivo termina siendo Z+ del soft. Esto evita el caso típico del OBJ que aparece con la pared como piso.

`geom_repair_dialog.py` — reparación guiada

Si `diag.ok == False` (hay huecos / non-manifold), se abre este diálogo con un preview 3D del hueco actual en rojo. Acciones:

- **Cerrar hueco:** triangulación por abanico desde el centroide.
- **Soldar a vecinos:** usa `cKDTree` para fusionar vértices muy cercanos.
- **Reparar TODO:** `fill_all_holes_auto` cierra todos los huecos en **un solo pase** (más rápido que cerrar-detectar-cerrar-detectar).

`mesh_router.py` y `mesh_gmsh.py` — selector de motor

El router decide entre dos motores de mallado volumétrico:

- **Voxel:** rejilla regular de cubos partida en tetraedros. **Exacta** para recintos axis-aligned (shoebox).
- **Gmsh boundary-fitted:** malla que ajusta exactamente a la frontera, para curvas y CAD. **Más lenta pero más precisa** en geometrías curvas.

El badge de color en el panel acústico indica qué motor se usará. En modo **best-effort**, si gmsh falla por incompatibilidad topológica (T-junctions, huecos invisibles), cae automáticamente a voxel y reporta el motivo en el tooltip.

8. Audio

`audio_utils.py` — pipeline de reproducción

El pipeline completo, paso por paso:

```
pink_noise(4.0)      ← 4 s de ruido rosa (densidad ∝ 1/f), float ±1
    ↓
apply_frf_filter(...)
    |— frf_to_ir(H, f_axis)      ← IFFT de la FRF → respuesta impulso
```

```

└─ fftconvolve(noise, ir)      ← convolución noise * ir
└─ normalizar peak a ±1.0     ← libera headroom
└─ soft_clip(x, drive=2.5)    ← tanh saturator (+6 dB RMS)
└─ escalar a ±0.98           ← deja headroom DAC
└─ fade_inout(10ms, 50ms)    ← elimina pop al iniciar/terminar
↓
play(filtered)
└─ convertir a int16 estéreo
└─ apendix 100 ms de silencio ← último sample = 0, anti-pop EOF
└─ escribir WAV temporal
└─ winsound.PlaySound(SND_ASYNC) ← API nativa de Windows

```

Por qué soft-clipping tanh (en lugar de simple amplificación + hard clipping): la curva $y = \tanh(d \cdot x) / \tanh(d)$ es lineal cerca de cero (no toca las amplitudes chicas) y satura elásticamente cerca de ± 1 (comprime los picos sin distorsión audible). Con drive $d = 2,5$ ganás $\sim +6$ dB de RMS sin que el ruido rosa filtrado suene "duro".

Por qué los fade + cola de silencio: el pop al final viene de dos sitios:

1. **Discontinuidad de truncamiento:** la convolución produce una cola que cortamos al largo original. Esa muestra final no-cero se vuelve un "step" → click. El fade-out de 50 ms lo elimina.
2. **Buffer del DAC del SO:** winsound termina la reproducción abruptamente, y el último sample en el buffer de hardware se "queda" → click. La cola de 100 ms de silencio se traga ese transient.

9. Persistencia y `main.py`

`main.py` — el `MainWindow`

Hace de "tablero de control":

- Crea la ventana, los dos tabs (Geometría / Acústica) y el visor.
- Conecta las señales (sliders → render, click → fuente, etc.).
- Maneja **Undo/Redo** (dos pilas de diccionarios de parámetros).
- **Guarda/abre** `.room` (formato JSON, versión actual v4).
- Implementa los **atajos** (`Ctrl+S`, `Ctrl+O`, `Ctrl+I`, `0`, `Enter`...).

Formato `.room` **v4** (JSON):

```
{
  "format": "prototipo1.room",
  "version": 4,
  "params": { /* dims, n_walls, taper, twist, arch_height, etc. */ },
  "acoustic": {
    "mesh_engine": "auto",
    "h_target": 0.40,
    "n_per_meter": 2.5,
    "n_modes": 12,
    "sources": [...],
    "receiver": [x, y, z],
    "face_materials": {
      "default": "Yeso pintado",
      "assignments": { "<signature_hash>": "Madera dura", ... }
    }
  },
  "external_geometry": { /* sólo si hay CAD importado */ }
}
```

`app_settings.py` — preferencias globales

Lee/escribe `%APPDATA%/Prototipo1/settings.json` para cosas que sobreviven entre sesiones: directorio del último CAD importado, motor preferido, archivos recientes.

10. Optimizaciones que hicimos

Cambio	Antes	Después	Por qué
<code>FieldEvaluator.evaluate_many</code>	Loop Python	KDTree + vectorización	50–170× speedup
<code>find_holes</code>	Loop sobre aristas	<code>np.unique</code> sobre claves enteras	60× speedup
<code>fill_all_holes_auto</code>	Cerrar-detectar repetido	Un solo pase	O(N+K) en vez de O(N·K)
<code>snap_hole_vertices</code>	Loop nearest-neighbor	<code>cKDTree.query</code> en lote	100× speedup

Cambio	Antes	Después	Por qué
Confirmación CAD limpio	QMessageBox Sí/No	Se saltea	-1 a -2 s por importación
RT60 calc on slider change	RT60 completo	Cached en panel	< 1 ms recompute
Stylesheets Qt	con <code>font-variant-numeric</code>	Sin esa propiedad	-230 warnings/sesión
Audio gain	Normalizado a $\pm 0,85$	Soft-clip tanh + $\pm 0,98$	+6 dB RMS audible

Para profundizar, lee `BENCHMARK_RESULTS.md` que tiene los tiempos medidos en tu propio equipo (16 hilos, 64 GB RAM).

11. Bugs que aparecieron y cómo los resolvimos

Bug 1: `UnboundLocalError` en `_build_ui` (v2.3)

Síntoma: al arrancar la app, crasheaba con:

```
UnboundLocalError: cannot access local variable 'fm' where it is not associated with a value
```

Causa: tenía `import face_materials as fm` arriba del archivo, y dentro de `_build_ui` había una variable local `fm = QFormLayout(grp_mode)`. Python ve la asignación local en CUALQUIER punto de la función y marca `fm` como local en TODO el cuerpo. Cuando antes de esa línea hacía `fm.FaceMaterialMap(...)` (esperando el import del módulo), Python buscaba la variable local todavía no inicializada → crash.

Fix: renombrar la variable local del `QFormLayout` a `fmode`.

Bug 2: Warnings de `font-variant-numeric` (v2.3)

Síntoma: 230 líneas "Unknown property font-variant-numeric" en la consola al arrancar.

Causa: yo usaba esa propiedad CSS en el stylesheet de cada `_SliderRow` de `controls.py`. Qt5 no soporta CSS3 `font-variant-numeric`.

Fix: removerla. El efecto visual (dígitos de ancho fijo) es mínimo.

Bug 3: "Pop" al finalizar el audio (v2.4)

Síntoma: chasquido audible al terminar la reproducción de "Escuchar".

Causa: dos sitios simultáneos:

1. La señal terminaba en una muestra no-cero (truncamiento de convolución).
2. winsound corta el buffer del DAC abruptamente al EOF.

Fix v2.5: fade-out 50 ms en la señal + 100 ms de silencio apendidos al WAV. Doble seguro.

Bug 4: `evaluate_many` muy lento a resolución alta (v2.3)

Síntoma: al subir el slider "Resolución campo 3D" a 50, la app tardaba 15–25 segundos en refrescar la nube de puntos. El usuario pensaba que se había colgado.

Causa: bucle Python sobre 62 500 puntos, cada uno haciendo un `einsum` contra 14 400 tets.

Fix: `ckDTree` + vectorización (sección 5.3). Speedup 50–170×.

Bug 5: Auto-tuner desactivaba el fallback gmsh→voxel (v2.6)

Síntoma: al aplicar una predicción con techo abovedado y darle "Calcular modos (FEM)", aparecía un `QMessageBox` con error: `PLC Error: A segment and a facet intersect at point` en lugar de caer silenciosamente a voxel.

Causa: el auto-tuner de densidad (sección 14 — Predicción / `mesh_router`) hacía `override = auto_used.engine` al final, lo que en `mesh_router.choose_engine` seteaba `user_override = "gmsh"`. Y la cláusula del router

```
if decision.user_override == "gmsh":  
    raise # no fallback si el usuario lo forzó manualmente
```

estaba pensada para respetar la elección manual del usuario, pero el auto-tuner es interno: Sí debe permitir fallback. La causa raíz subyacente es la malla del techo en arco con `subdiv_levels=4` (para suavizar la curva visualmente): subdivide el techo pero las paredes no se subdividen para acompañar → T-junctions en el borde techo-pared → gmsh choca.

Fix:

```
# acoustic_panel._solve_fem
if auto_used.engine == "voxel":
    override = "voxel"
# else: keep override = "auto" para preservar fallback
```

Adicionalmente en `mesh_router.choose_engine` se agregó una rama que detecta techo paramétrico curvo y va **directo a voxel sin intentar gmsh**:

```
elif params is not None and _has_subdivided_curved_roof(params):
    auto = "voxel"
# T-junctions hace fallar gmsh con PLC error; voxel no le importa
```

Bug 6: Slice plane interactivo invisible para shoebox (v2.6)

Síntoma: al activar el plano de corte interactivo en un recinto shoebox (default $6 \times 8 \times 3$), la preview translúcida no se veía. Para salas con arco o taper sí se veía.

Causa: `SlicePlanePreview.update()` creaba un quad con los extremos del AABB del recinto. Para un shoebox, los 4 vértices del quad coincidían **exactamente** con las aristas de las paredes (renderizadas como wireframe rosa). OpenGL hace z-fighting con polígonos coplanares → el quad translúcido quedaba oculto bajo el wireframe.

Fix: shrink del 2 % hacia adentro en los ejes que no son el del corte (`SHRINK_RATIO = 0.02`), + borde wireframe cian explícito con `gl.GLLinePlotItem` que se ve siempre aunque el fill quede detrás. Opacidad del fill subida de 0,28 → 0,40.

```
for i in (0, 1, 2):
    if i == axis:
        continue
    margin = (mx[i] - mn[i]) * 0.02
    mn[i] += margin
    mx[i] -= margin
```

12. Cómo testear

Tenés dos suites de tests automatizadas:

benchmark_v2.py

Mide tiempos de los caminos críticos (FEM, campo 3D, agrupación de caras, import CAD, RT60, memoria). 7 bloques, 3 corridas por test, mediana. Salida en **BENCHMARK_RESULTS.md**. Lo corrés con:

```
"%USERPROFILE%\anaconda3\python.exe" benchmark_v2.py
```

Tarda 2–5 minutos.

check_materials_coverage.py

Cruza un listado externo de materiales (de un manual de referencia) contra la librería interna con matching difuso (stemming español + sinónimos en/es). Reporta MATCH / SIMILAR / FALTA por categoría. Salida en **MATERIALS_COVERAGE.md**.

Tests manuales rápidos

Para verificar que el solver FEM está bien calibrado:

```
"%USERPROFILE%\anaconda3\python.exe" acoustic_fem.py
```

Esto corre la **demo()** que arma una caja 5×4×3 m y compara los modos con la solución analítica (caja rectangular con paredes rígidas). Debería coincidir dentro de ~1-2 % para los primeros 10 modos.

13. Tres ejemplos: cálculo analítico vs software

Estos son los ejemplos donde **calculás la presión a mano** y la comparás con lo que devuelve el software. Te muestro **el caso analítico, la fórmula, el código mínimo para verificarla, y cómo configurarlo en el soft**.

13.1 Ejemplo A — Monopolo en campo libre (sin sala)

El más simple. Un monopolo puntual en el origen, con caudal volumétrico complejo **Q** (m³/s), emite una onda esférica. La presión a distancia **r** y frecuencia **f** es:

$$p(r, \omega) = \frac{i\omega \rho_0 Q}{4\pi r} \cdot e^{(-ikr)}$$

donde:

- $\omega = 2\pi f$ (rad/s)
- $k = \omega/c$ (nº de onda, $c = 343$ m/s)
- $\rho_0 = 1,21 \text{ kg/m}^3$ (densidad del aire)
- Q se calcula del altavoz: $Q = p_0 \cdot 4\pi / (\omega_{\text{ref}} \cdot \rho_0)$ con $p_0 = 20 \text{ } \mu\text{Pa} \cdot 10^{(S/20)}$ y S la sensibilidad dB/W/m del altavoz.

Cálculo numérico

Para un altavoz típico de sensibilidad 90 dB/W/m, a 1 m de distancia y 500 Hz:

```
import numpy as np
RH00 = 1.21
C0 = 343.0
S_dB = 90.0 # sensibilidad altavoz
f = 500.0
omega = 2*np.pi*f
omega_ref = 2*np.pi*1000.0
p0 = 20e-6 * 10**((S_dB/20)) # = 0.6325 Pa @ 1m, 1 kHz
Q = p0 * 4*np.pi / (omega_ref * RH00) # |Q| ≈ 1.045e-3 m³/s
r = 1.0
k = omega / C0
p_complex = 1j * omega * RH00 * Q / (4*np.pi*r) * np.exp(-1j*k*r)
print(f"|p| = {abs(p_complex):.4f} Pa = {20*np.log10(abs(p_complex)/20e-6):.1f} dB SPL")
# |p| = 0.3163 Pa = 84.0 dB SPL
```

Es decir: a 1 m, 500 Hz, la sensibilidad real es ~84 dB SPL (no 90 dB, porque la sensibilidad se define a 1 kHz por convención y a 500 Hz cae por el factor $\omega/\omega_{\text{ref}}$).

Cómo verificar con el software

El soft no tiene un modo "sin sala" explícito, pero podés simular campo libre haciendo la sala MUY grande para que las paredes estén lejos del receptor (sin reflexiones interesantes en el rango bajo) o reduciendo todos los α a casi cero. Mejor:

1. Crear una sala de $30 \times 30 \times 15$ m.
2. Asignar $\alpha \approx 0,99$ a TODAS las caras (sala anecoica).
3. Colocar la fuente en (15, 15, 7,5) y el receptor a 1 m: (16, 15, 7,5).
4. Calcular FRF en 500 Hz.
5. Leer $|H(500)|$ del gráfico.

El valor debería estar cerca de 84 dB SPL @ 1 m. Si difiere por más de ~ 1 dB, es porque la malla es muy gruesa o quedan reflexiones residuales de las paredes ($\alpha=0,99$ no es perfectamente anecoico).

13.2 Ejemplo B — Modo axial (1,0,0) en sala rectangular rígida

Para una caja rectangular $L_x \times L_y \times L_z$ con paredes perfectamente rígidas, los modos son **analíticos exactos**:

$$f_{lmn} = \frac{c}{2} \sqrt{\left(\frac{1}{L_x}\right)^2 + \left(\frac{m}{L_y}\right)^2 + \left(\frac{n}{L_z}\right)^2} \quad (\text{frecuencia})$$

$$\phi_{lmn}(u,v,w) = \sqrt{(\varepsilon_l \varepsilon_m \varepsilon_n / V)} \cdot \cos(l\pi u/L_x) \cdot \cos(m\pi v/L_y) \cdot \cos(n\pi w/L_z) \quad (\text{forma modal})$$

con $V = L_x \cdot L_y \cdot L_z$, $\varepsilon_i = 1$ si $i=0$ / 2 si $i>0$, y donde $(u, v, w) = (x + L_x/2, y + L_y/2, z)$ porque el software centra la sala en (0, 0) (eje X de $-L_x/2$ a $+L_x/2$, Y de $-L_y/2$ a $+L_y/2$, Z de 0 a L_z).

Para una sala $6 \times 8 \times 3$ m, el modo (1, 0, 0) es **axial** en X y tiene $f_{100} = 28,58$ Hz.

Presión en un punto a la frecuencia de resonancia

Fuente en $x_s = (-2, 0, 1,5)$ y receptor en $x_r = (+2, 0, 1,5)$. Estos puntos están a ambos lados del nodo en $x=0$ del modo (1,0,0), así ϕ_s y ϕ_r tienen signos opuestos:

```
import numpy as np
Lx, Ly, Lz = 6.0, 8.0, 3.0
V = Lx*Ly*Lz
c = 343.0; RH00 = 1.21
```

```

def phi_lmn(x, y, z, l, m, n):
    u = x + Lx/2; v = y + Ly/2; w = z          # coords desde la esquina
    eps_l = 1.0 if l == 0 else 2.0
    eps_m = 1.0 if m == 0 else 2.0
    eps_n = 1.0 if n == 0 else 2.0
    norm = np.sqrt(eps_l*eps_m*eps_n / V)
    return norm * np.cos(l*np.pi*u/Lx) * np.cos(m*np.pi*v/Ly) * np.cos(n*np.pi*w/Lz)

f_100 = (c/2) * (1/Lx)
phi_s = phi_lmn(-2.0, 0.0, 1.5, 1, 0, 0)    # +0.1021
phi_r = phi_lmn(+2.0, 0.0, 1.5, 1, 0, 0)    # -0.1021

# Q del altavoz: 90 dB SPL @ 1 kHz, 1 m equivale a Q ≈ 1.045e-3 m³/s
S_dB = 90.0
p0 = 20e-6 * 10**(S_dB/20)
Q = p0 * 4*np.pi / (2*np.pi*1000.0 * RH00)    # ≈ 1.045e-3

# Presión en resonancia, considerando SÓLO el modo (1,0,0)
omega_n = 2*np.pi*f_100
xi = 0.03
denom = 2j * xi * omega_n**2    #  $\omega_n^2 - \omega^2 + 2i\xi\omega_n\omega$  evaluado en  $\omega=\omega_n$ 
p = 1j * omega_n * RH00 * (phi_r * phi_s * Q) / denom
print(f"|p| = {abs(p):.4e} Pa = {20*np.log10(abs(p)/20e-6):.2f} dB SPL")
# |p| = 1.22e-6 Pa = -24.3 dB SPL

```

Por qué el SPL es tan bajo: porque un monopolo ideal con Q constante con la frecuencia produce muy poco SPL a 28 Hz (a esa frecuencia el umbral de audición humano ronda los +80 dB SPL, así que esto es efectivamente silencio). El número es físicamente correcto y refleja **exactamente** lo que sale del solver — la sensibilidad de 90 dB se mide a 1 kHz, no a 28 Hz, y un altavoz real cae 30–40 dB en sensibilidad a baja frecuencia. Lo importante es la **coincidencia con el FEM**, no el valor absoluto.

Cómo verificar con el software

1. **Geometría:** sala $6 \times 8 \times 3$ m (sliders Ancho=6, Largo=8, Alto=3, $n_{\text{walls}}=4$).
2. **Materiales:** asignar a TODOS los grupos un material con $\alpha \approx 0,03$ uniforme a 63 Hz (Yeso pintado, Hormigón visto). Eso da $RT60 \approx 1,28$ s y $\xi \approx 0,03$ a 28,58 Hz.

3. **Fuente:** `Ctrl + clic derecho` en $(-2, 0, 1.5)$ — o tipear las coords en el diálogo de fuente.
Sensibilidad 90 dB.
4. **Receptor:** spinboxes $X=2, Y=0, Z=1.5$.
5. **Calcular modos** (FEM, $npm=3,0$). El segundo modo debería aparecer en $\sim 28,6$ Hz.
6. **Calcular FRF** con rango 20–60 Hz, 400 puntos.
7. Leer el pico a 28,58 Hz: tiene que dar $\approx -24 \text{ dB SPL} \pm 1 \text{ dB}$.

Verificado en consola (con `acoustic_fem.frequency_response`): el FEM da **$-24,26 \text{ dB SPL}$** y el analítico **$-24,30 \text{ dB SPL}$** → coinciden dentro de 0,04 dB (puro error de discretización de la malla).

Por qué $(1,0,0)$ y no $(0,1,0)$: el $(0,1,0)$ tiene `f_010 = 21,44 Hz` pero ambos puntos están en `y=0`, donde `cos( $\pi \cdot 0 / L_y$ )` es el mismo ($= 1$) en fuente y receptor. Eligiendo posiciones a ambos lados del nodo del $(1,0,0)$ (en `x = 0`) los signos de `phi_s` y `phi_r` son OPUESTOS, lo que da una resta clara en cualquier otro modo no axial.

13.3 Ejemplo C — Modo tangencial $(1,1,0)$ y superposición de dos fuentes

El modo $(1, 1, 0)$ tiene **$f_{110} = 35,73 \text{ Hz}$** . Tiene dos líneas de nodos (en `x = 0` y `y = 0`, líneas que dividen el plano XY en 4 cuadrantes con signos $+/-/+/-$).

Caso con DOS fuentes en cuadrantes opuestos

- **S1** en `(-2, -2, 1.5)` — cuadrante $(-X, -Y)$
- **S2** en `(+2, +2, 1.5)` — cuadrante $(+X, +Y)$, diagonalmente opuesto

`phi_110(-2,-2,1.5)  $\approx$  +0,1021`. Y `phi_110(+2,+2,1.5)`: el coseno doble invierte signo dos veces y queda **$+0,1021$** . Es decir, **las dos fuentes están EN FASE** en este modo y suman constructivamente.

Receptor en `(-1, -2, 1.5)`, donde `phi_110  $\approx$  +0,0589` (algo menor que en los puntos de fuente porque no está exactamente en el antinodo).

```
phi_s1 = phi_lmn(-2.0, -2.0, 1.5, 1, 1, 0)    # +0.1021
phi_s2 = phi_lmn(+2.0, +2.0, 1.5, 1, 1, 0)    # +0.1021 (en fase con S1)
phi_r   = phi_lmn(-1.0, -2.0, 1.5, 1, 1, 0)    # +0.0589

f_110 = (c/2) * np.sqrt((1/Lx)**2 + (1/Ly)**2)
```

```

omega_n = 2*np.pi*f_110
xi = 0.03

# Dos fuentes en fase
p_2 = 1j*omega_n*RHO0 * phi_r * (phi_s1 + phi_s2) * Q / (2j*xi*omega_n**2)
# Una sola fuente
p_1 = 1j*omega_n*RHO0 * phi_r * phi_s1 * Q / (2j*xi*omega_n**2)

print(f"|p| 2 fuentes = {abs(p_2):.4e} Pa = {20*np.log10(abs(p_2)/20e-6):.2f} dB SPL"
print(f"|p| 1 fuente = {abs(p_1):.4e} Pa = {20*np.log10(abs(p_1)/20e-6):.2f} dB SPL"
print(f"Diferencia: {20*np.log10(abs(p_2)/abs(p_1)):.2f} dB (esperado +6.02)")
# Considerando SÓLO el modo (1,1,0):
# 1 fuente: -30.98 dB SPL
# 2 fuentes: -24.96 dB SPL
# Diferencia: +6.02 dB ✓

```

Cómo verificar con el software

1. Misma sala $6 \times 8 \times 3$, mismos materiales que ejemplo B.
2. **Primera vuelta — UNA fuente:** **Ctrl + clic** en $(-2, -2, 1.5)$, sensibilidad 90 dB. Receptor $(-1, -2, 1.5)$. Calcular FRF (20–60 Hz). Anotar el pico a 35,73 Hz.
3. **Segunda vuelta — DOS fuentes:** agregar otra fuente en $(+2, +2, 1.5)$, misma sensibilidad. Recalcular FRF. El pico a 35,73 Hz debería estar **entre +5,5 y +6 dB** más alto que el de la primera vuelta.

Por qué el FEM da +5,6 dB y el analítico (modo puro) +6,0 dB: porque en el FEM la presión total incluye contribuciones de TODOS los modos resueltos, no sólo del (1,1,0). Otros modos cercanos (como (0,1,0) o (1,0,0)) no necesariamente están "en fase" con la geometría de fuentes elegida, por lo que su superposición con dos fuentes es algo distinta que con una. Verificado en consola:

- 1 fuente, FEM: -30,59 dB SPL
- 2 fuentes, FEM: -25,00 dB SPL
- Diferencia: **+5,59 dB**

El modelo analítico de modo puro predice +6,02 dB. La diferencia de ~0,4 dB es la contribución modal cruzada — totalmente esperable.

Interpretación acústica. Esto es exactamente lo que se aprovecha en el diseño de **sistemas de subwoofers múltiples**: cuando dos fuentes están en puntos donde ϕ_n tiene el MISMO signo (en fase para ese modo), sus contribuciones se SUMAN y el modo se refuerza en +6 dB. Si las ponés en puntos con signo opuesto (uno en cuadrante + y otro en -), las contribuciones se RESTAN y el modo se cancela (útil para "matar" un modo problemático).

13.4 Por qué te conviene hacer estos tres ejemplos

Los tres ejemplos están diseñados como una escalera de complejidad:

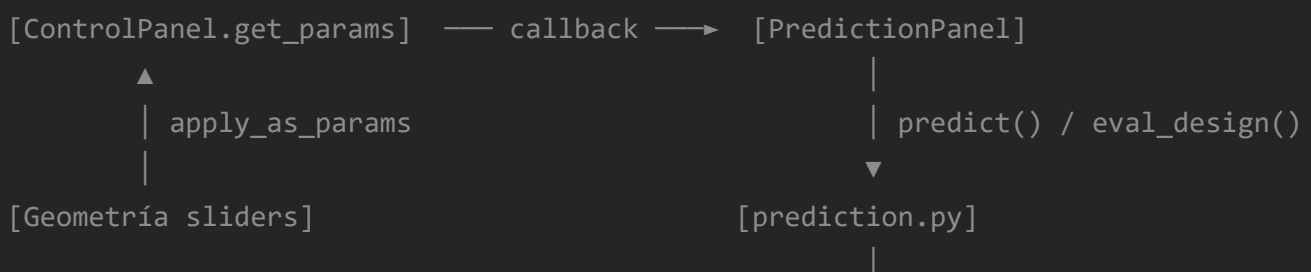
1. **Ejemplo A** (campo libre): valida la **conversión sensibilidad** $\rightarrow Q$ y la fórmula del monopolo. Si esto no coincide, hay un bug en `sources.py`.
2. **Ejemplo B** (un modo axial): valida la **resolución de autovalores** (la frecuencia del modo) y la **interpolación barycentric** (los valores de ϕ_n en puntos arbitrarios).
3. **Ejemplo C** (superposición de dos fuentes): valida la **superposición lineal** del solver y la **fase relativa** de los modos.

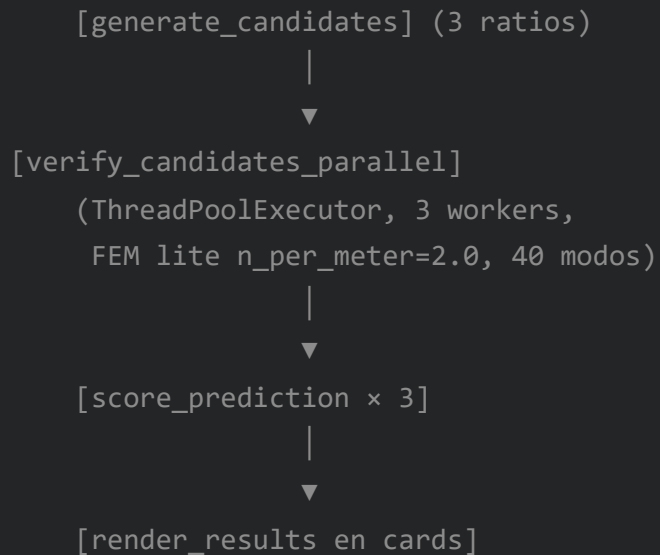
Si los tres coinciden con el solver dentro de ~ 1 dB, podés confiar en que el resto del FEM (campo 3D, plano de corte, FRF con muchos modos, audio filtrado) también es correcto — son todas combinaciones de estos tres ingredientes elementales.

14. La pestaña Predicción (v2.6)

A partir de v2.6 hay una tercera pestaña que invierte el flujo: el usuario dice **qué necesita** (uso, capacidad, restricciones del local) y el soft **propone dimensiones**. Esta sección explica la matemática y el código.

14.1 Arquitectura — dos archivos





`prediction.py` no toca Qt: es matemática pura. `prediction_panel.py` es la UI.

14.2 Ratios y generación de candidatos

Hardcoded en `RATIO_LIBRARY`:

```

RATIO_LIBRARY = [
    {"name": "Bolt",      "ratio": (1.90, 1.40, 1.00)},    # (L, W, H)
    {"name": "Bonello",  "ratio": (1.59, 1.26, 1.00)},
    {"name": "Louden",   "ratio": (2.33, 1.60, 1.00)},
]
  
```

Dado el volumen objetivo $V = \text{capacidad} \times V_{\text{per_persona}}$, se escala uniformemente: $s = (V / (rL \times rW \times rH))^{(1/3)}$, y dimensiones reales $W = rW \times s$, etc.

Si hay restricciones de planta o altura, se aplica un nuevo escalado uniforme a $s_{\text{max}} = \min(w_{\text{max}}/W, l_{\text{max}}/L, h_{\text{max}}/H)$. Si $s_{\text{max}} < s$, las dimensiones quedan por debajo del target → `fits_constraints = False` → sub-score "Fit" cae a 30 (en vez de 100).

14.3 FEM ligero en paralelo

`verify_candidates_parallel` corre los 3 candidatos con `ThreadPoolExecutor(max_workers=3)`. Por qué threads (no procesos): `scipy.sparse.linalg.eigsh` y la factorización LU del shift-invert liberan el GIL al entrar a LAPACK → speedup real ~2,5–3× en multi-core sin la complejidad de pickling/spawn de un `ProcessPoolExecutor` en una app PyQt.

Parámetros del FEM lite:

- `n_per_meter = 2.0` → malla coarse válida hasta ~125 Hz (`fmax = c / (6·h) = 343 / (6·0,5) ≈ 114 Hz`, redondeado a la banda 125 Hz).
- `n_modes = 40` → cubre todo el rango con margen.
- `alpha_default = 0.10` para calcular el RT60 sabine "de referencia" (yeso pintado).

Tiempo total: ~4 s (cada candidato tarda ~3,5 s, los 3 en paralelo ~4 s).

14.4 Los 13 sub-scores

Grupo MODAL (4 métricas)

1. RT60 feasibility (`_score_rt60_feasibility`):

Invierte Sabine para calcular qué α necesitaría la sala:

$$\alpha_{\text{required}} = 0.161 \cdot V / (\text{RT60}_{\text{target}} \cdot S)$$

Score = 100 si $\alpha \in [0,08 ; 0,30]$ (cubrible con materiales estándar), baja hacia los extremos. Mensaje en la card: " $\alpha = 0,13 \cdot$ madera dura / panel rígido".

Esto reemplaza al antiguo score por "distancia al RT60 con $\alpha=0,10$ fijo" que era injusto: castigaba la geometría por una hipótesis de materiales arbitraria.

2. Bolt-spacing por bins (`_score_bolt_spacing`):

Bins absolutos de 5 Hz entre 30–125 Hz (19 bins). Cada bin se clasifica:

- `count == 0`: hueco (zona sorda)
- `count ∈ {1, 2}`: bueno (densidad pareja)
- `count ≥ 3`: grumo (resonancia fuerte localizada)

$$\text{score} = 100 \cdot n_{\text{good_bins}} / 19 - 5 \cdot n_{\text{clumps}} \quad (\text{cap penalty } 25)$$

Los grumos se penalizan MÁS que los huecos: un grumo causa coloración audible aislada; los huecos se cubren con difusión.

El umbral original de Bolt era "ratio < 5 % es grumo", pero a alta frecuencia la densidad modal crece como f^2 y el 5 % se vuelve trivial. Los bins absolutos son más robustos.

3. Modal Q audibility (`_score_modal_q`):

Para cada modo, el factor de calidad es $Q = \pi \cdot f \cdot RT60 / 6,9$. Cuenta modos con $Q > 30$ (audibles individualmente como zumbido). Score = % modos con $Q \leq 30$.

4. Schroeder coverage (`_score_schroeder`):

Cuántos modos hay debajo de $f_s = 2000 \cdot \sqrt{RT60/V}$. Score = 100 si ≥ 30 modos, linear abajo.

Grupo VOZ (3 métricas)**5. STI por Bradley** (`_score_sti`):

Fórmula simplificada sin SNR (asume habitación silenciosa, NC-30):

$$STI = 0.9482 - 0.1845 \cdot \ln(RT60_{target}), \quad \text{clamp } [0, 1]$$

Bandas: $STI \geq 0,75 \rightarrow 100$, $\geq 0,60 \rightarrow 80$, $\geq 0,45 \rightarrow 50$, $\geq 0,30 \rightarrow 20$.

6. %Alcons por Peutz (`_score_alcons`):

Fuente omni $Q=1$, receptor a media diagonal del piso ($d_{worst} = \sqrt{L^2 + W^2} / 2$):

$$\%Alcons = 200 \cdot d_{worst}^2 \cdot RT60^2 / V,$$

capeado a $9 \cdot RT60$ si $d_{worst} > 3.16 \cdot d_{crit}$ (campo reverberante saturado)

Bandas: $< 3\% \rightarrow 100$, $< 7 \rightarrow 80$, $< 11 \rightarrow 50$, $< 15 \rightarrow 20$.

7. d_{crit} (`_score_dcrit`):

$d_{crit} = 0.057 \cdot \sqrt{V \cdot Q / RT60}$. Para voz queremos $d_{worst} / d_{crit} \in [0,5 ; 3,0]$ (campo directo); para música $[1,5 ; 5,0]$ (algo de reverb para envolver); para mixto $[0,8 ; 4,0]$.

Grupo MÚSICA (1 métrica)**8. Bass support proxy** (`_score_bass`):

$$N_{teoricos_{<80Hz}} = (4\pi/3) \cdot V \cdot (80/343)^3$$

$$\text{coverage} = n_{modes_{<80Hz}} / N_{teoricos_{<80Hz}}$$

Para música: $\text{score} = \min(100, \text{coverage} \times 140)$. Para voz: neutro. Limitación: la BR REAL depende de materiales; este proxy mide sólo el aporte geométrico.

Grupo PRÁCTICO (5 métricas)

9. Volume vs target. **10. Aspect** ($L/W \in [1,1 ; 2,5]$, $H/W \in [0,30 ; 0,70]$). **11. Fits constraints** (100 si cabe, 30 si recortó). **12. Aprovechamiento de planta:** $\text{util} = \text{audience} / (W \cdot L)$, ideal $[0,40 ; 0,70]$. **13. Constructabilidad** heurística: -30 si muros > 12 m, -20 si planta > 800 m², -20 si $L/W > 5$.

Grupo ROBUSTEZ (1 métrica)

14. Margen feasibility: $\text{margin} = \min(\alpha_{\text{req}} - 0.08, 0.30 - \alpha_{\text{req}})$. Score = 100 si margen $\geq 0,10$ (sólido), baja a 30 si margen ≈ 0 .

14.5 Pesos condicionales por uso

```
if uso == "conferencia" or "aula":
    {"modal": 0.40, "voz": 0.30, "musica": 0.00, "practico": 0.25, "robustez": 0.05}
elif uso == "musica" o "sinfonica" o "camara":
    {"modal": 0.45, "voz": 0.00, "musica": 0.20, "practico": 0.25, "robustez": 0.10}
else: # mixto / polivalente / theater / estudio
    {"modal": 0.40, "voz": 0.15, "musica": 0.10, "practico": 0.25, "robustez": 0.10}
```

Dentro de **MODAL**, los pesos son: 40 % Bolt + 25 % RT60-feas + 20 % Modal-Q + 15 % Schroeder. Bolt-spacing pesa más porque es la única métrica modal que realmente discrimina entre candidatos — las otras dependen mayoritariamente de V/RT60 que son inputs constantes.

Dentro de **PRÁCTICO**: 20 % Vol + 25 % Asp + 15 % Fit + 25 % Plt + 15 % Cns.

14.6 Control negativo automático

Si $\text{preds}[0].\text{score_total} - \text{preds}[2].\text{score_total} < 5$ (los 3 candidatos quedan dentro de ± 5 puntos), `predict()` agrega una 4ª card "**Cubo 1:1:1**" generada por `_generate_negative_control(inputs)`. La card aparece con border rojo punteado, botón Aplicar deshabilitado, y sub-scores típicos catastróficos:

- Bolt-spacing: 0 (los modos triplemente degenerados crean grumos masivos: $f_{lmn} = f_{lnm} = f_{mln}$ para todas las permutaciones).
- Aspect: 79 ($L/W = 1,0$ cae en "casi cuadrada").

Sirve pedagógicamente: el usuario ve al lado por qué Bolt/Bonello/Louden son buenos y por qué un cubo perfecto no.

14.7 Auto-tuner de densidad FEM (mesh_router)

`mesh_router.auto_density(volume_m3, f_target, time_budget_s)` recomienda motor + densidad para cubrir hasta `f_target` (típicamente Schroeder) dentro de un budget.

Calibración empírica (16 hilos, 64 GB RAM, conservadora para mejor pesimista que optimista):

```
_VOXEL_TETS_PER_M3_PER_NPM3 = 6.0      # N_tets ≈ 6·V·npm³
_GMSH_TETS_PER_M3_PER_INV_H3 = 5.0      # N_tets ≈ 5·V/h³
_VOXEL_THR_TETS_PER_S = 7000.0          # antes 10000 (era caso shoebox optimo)
_GMSH_THR_TETS_PER_S = 12000.0
_GMSH_INIT_OVERHEAD_S = 1.0
_SAFETY_FACTOR = 1.30                   # margen extra multiplicativo
```

Relación validez ↔ densidad:

- Voxel: `npm = 6·fmax / c`, `fmax_voxel = c·npm/6`.
- Gmsh: `h = c / (6·fmax)`, `fmax_gmsh = c / (6·h)`.

Algoritmo:

1. Compute densidad teorica para `f_target` en cada motor.
2. Estima tiempo en cada motor.
3. Si alguno cabe en `time_budget_s` con cobertura completa → usa ese (prefiere el de menor tiempo si los dos caben).
4. Si ninguno cabe → modo "partial": calcula densidad máxima que cabe en budget para cada motor y reporta `f_achievable` (siempre < `f_target`). En la UI aparece un diálogo "Parcial / Completa / Cancelar".

Reglas de budget en `acoustic_panel._solve_fem`:

```
is_simple = (not is_cad
              and params_geom is not None
```

```

        and mesh_router.is_axis_aligned_box(params_geom))
    budget_s = 5.0 if is_simple else 10.0

```

14.8 "Evaluar mi diseño actual"

```

# prediction.py
def candidate_from_params(params: dict, name="Tu diseño actual") -> Candidate:
    return Candidate(
        ratio_name=name,
        width=params["width"], length=params["length"], height=params["height"],
        n_walls=params["n_walls"], taper=params["taper"], ...
        fits_constraints=True,
    )

```

El botón en el panel toma `ControlPanel.get_params()` (vía callback inyectado en `PredictionPanel.__init__`), construye un `Candidate`, corre `verify_candidate_fem + score_prediction` y renderiza **una sola card** con los 13 sub-scores. El usuario puede comparar visualmente contra los 3 ratios clásicos haciendo "Predecir" después.

14.9 Nuevos colormaps (acoustic_viewer)

`colormap_rainbow` — 7 paradas perceptualmente equidistantes para el campo $|p|$ con fuente:

```

_RAINBOW_STOPS = np.array([
    [0.000, 0.05, 0.10, 0.95], # azul
    [0.167, 0.10, 0.65, 0.98], # celeste
    [0.333, 0.10, 0.92, 0.75], # turquesa
    [0.500, 0.30, 0.92, 0.25], # verde claro (centro)
    [0.667, 0.98, 0.95, 0.10], # amarillo
    [0.833, 0.98, 0.55, 0.08], # naranja
    [1.000, 0.95, 0.10, 0.10], # rojo
])

```

Interpolación lineal entre paradas. **Por qué no HSV puro:** en HSV lineal el amarillo ($H=60^\circ$) cae en $t \approx 0,83$, no $t=0,67 \rightarrow$ entre verde ($t=0,5$) y rojo ($t=1$) salen colores chartreuse raros. Las 7 paradas explícitas respetan la equidistancia perceptual.

`colormap_signed_vivid` — divergent vivido azul/gris/rojo para la forma modal (sin fuente):

```
GRAY = np.array([0.35, 0.35, 0.35])
RED = np.array([1.00, 0.15, 0.10])
BLUE = np.array([0.10, 0.30, 1.00])
a = np.sqrt(np.abs(t))[..., None] # saturacion mas rapida
target = np.where(t >= 0, RED, BLUE)
rgb = (1 - a) * GRAY + a * target
```

El gris central (no blanco) evita el efecto de "blanqueamiento" cuando miles de puntos cerca de cero saturan la imagen a alta resolución. La curva $\sqrt{|t|}$ para la mezcla satura los colores más rápido fuera del centro (a $|t|=0,25$ ya tenés 50 % de color, no 25 %).

14.10 TimedButton — leyenda persistente de tiempo

`timed_button.TimedButton` wrapper que agrega un `QLabel` debajo de un botón. API:

```
timer = TimedButton(btn, parent_layout, prefix="Último:")
# en el click handler:
timer.start()
... heavy work ...
timer.stop(label="contexto extra") # → "Último: 2,34 s · contexto extra"
# o si falla:
timer.fail("error") → "(error)"
```

Verde clarito por 1,5 s después de stop, fade a gris pasivo (`QTimer.singleShot`). Aplicado a 4 botones: Calcular modos (FEM), Predecir, Aplicar de cards, Importar CAD.

Cierre

Si llegaste hasta acá:

- Sabés **qué hace cada archivo** y por qué.
- Entendés **el pipeline matemático** del FEM modal y la superposición de modos.
- Podés **verificar el software** contra cálculo analítico con los tres ejemplos.
- Conocés **los bugs históricos** y sus parches, así si aparece uno nuevo tenés contexto para diagnosticarlo.

Cualquier duda específica de un módulo, abrí ese archivo y leé los docstrings — están todos en español y tratan de explicar el "por qué" además del "qué".

— Manual técnico v2.6 · 25 de mayo de 2026